

29) Master Theorem for Recurrence $T(n) = aT(n/b) + f(n)$.

MASTER THEOREM. $T(n) = aT(n/b) + f(n)$

case 1
 $f(n) = O(n^{\log_b a - \epsilon})$
 for some $\epsilon > 0$

Condition
 $f(n)$ grows asymptotically slower than $n^{\log_b a}$

Complexity Result
 $T(n) = \Theta(n^{\log_b a})$

Intuition
 The recursive work (cost at the leaves) dominates the combine (non-recursive) cost is relatively small.

case 2
 $f(n) = \Theta(n^{\log_b a} \log^k n)$
 for some $k \geq 0$

Condition
 $f(n)$ grows at the same rates as $n^{\log_b a}$ up to a polylog factor

Equal Growth
 $f(n)$ and $n^{\log_b a}$ are the same order differing by only a polylogarithmic factor $\log^k n$.

Complexity Result
 $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$

Intuition
 The recursive work and the combine works contribute equally at all levels of the recursion tree, leading to an extra $\log n$ factor

case 3
 $f(n) = \Omega(n^{\log_b a + \epsilon})$
 for some $\epsilon > 0$.

Condition
 $f(n)$ grows asymptotically faster than $n^{\log_b a}$.

Regularity condition
 There exist constant $c < 1$ and n_0 such that for all $n \geq n_0$,
 $a f(n/b) \leq c f(n)$

Complexity Result
 $T(n) = \Theta(f(n))$

Intuition
 The non-recursive work at the current level dominates the total cost.

39) Substitution Method.

Substitution method
Choose a guess for $T(n)$ (upper or lower bound)



Assume solution
Assume
 $T(n) \leq c \cdot g(n)$
(or)
 $T(n) \geq c \cdot g(n)$
for some $c > 0$



Expand Recurrence
Substitute the assumed solution into the recurrence and simplify.



Verified complexity
Conclude
 $T(n) = O(g(n))$
(or)
 $T(n) = \Omega(g(n))$



Prove by Induction
Show that the assumed solution holds for all $n \geq n_0$.



Base case
Verify the assumed solution holds for small values (usually $n = n_0$).

Induction Hypothesis.
Assume the solution holds for all $k < n$ and use it to prove for n .